ΣΥΣΤΗΜΑΤΑ ΔΙΑΧΕΙΡΙΣΗΣ ΚΑΙ ΑΝΑΛΥΣΗΣ ΔΕΔΟΜΕΝΩΝ

Εαρινό εξάμηνο 2024-2025

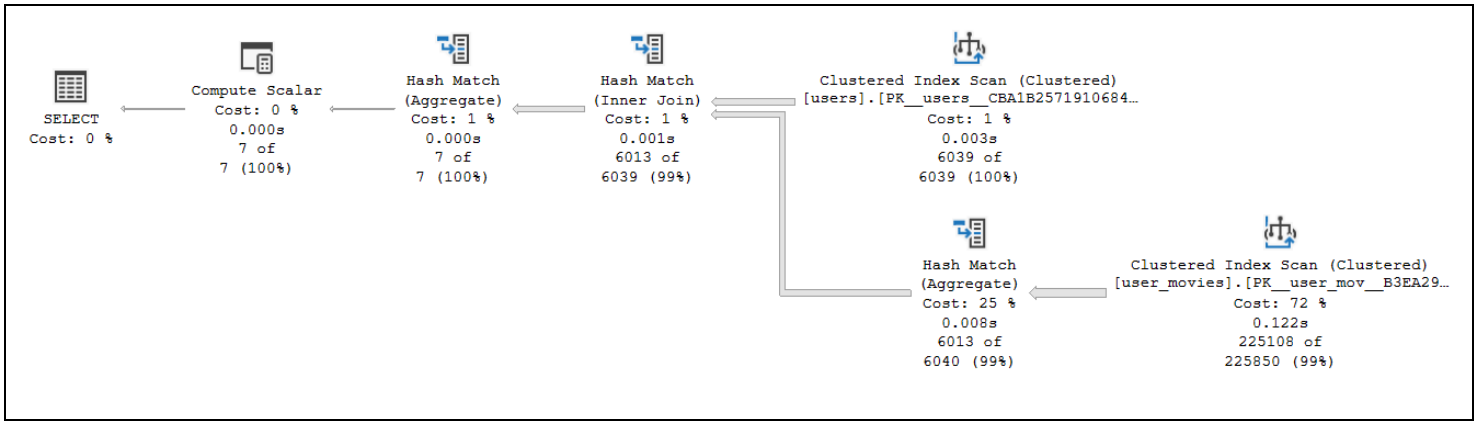
Βάση Δεδομένων Κινηματογραφικών Ταινιών

Ζήτημα Πρώτο

Query:

```
SELECT age, count(user_movies.userid)
FROM users
INNER JOIN user_movies ON users.userid = user_movies.userid
WHERE rating = 5
GROUP BY age;
```

Αρχικό Πλάνο Εκτέλεσης:



Αρχικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
user_movies	1	2601	2	2603
users	1	29	1	34

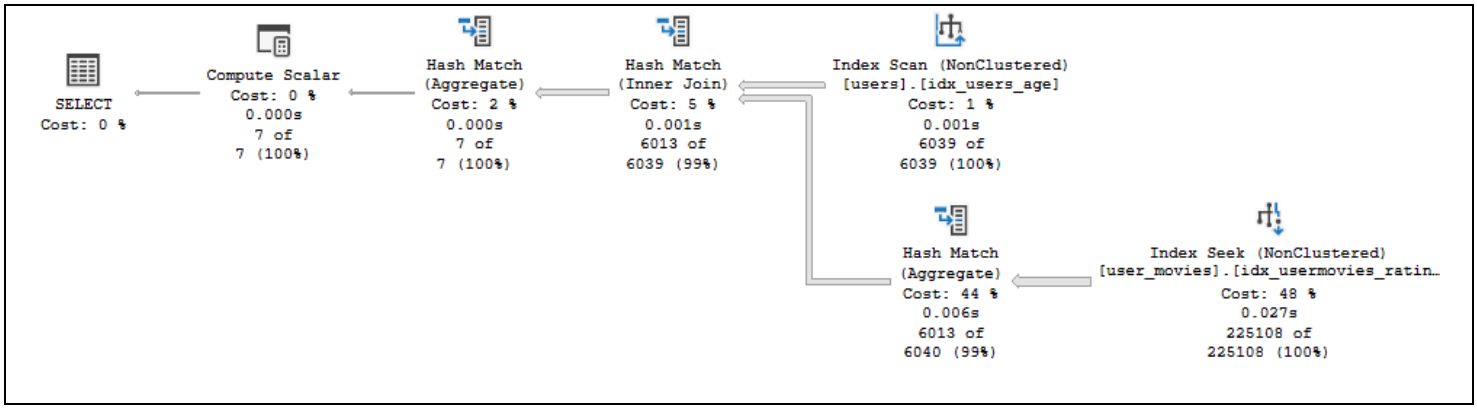
Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	109	175

Ευρετήρια:

```
CREATE INDEX idx_usermovies_rating
ON user_movies(rating);

CREATE INDEX idx_users_age
ON users(age);
```

Τελικό Πλάνο Εκτέλεσης:



Τελικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
user_movies	1	508	3	510
users	1	13	1	18

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	16	74

Συμπεράσματα:

Δημιουργήσαμε ευρετήριο με κλειδί το πεδίο rating, καθώς χρησιμοποιείται για φιλτράρισμα (WHERE), στο επερώτημα και αυτό είχε ως αποτέλεσμα σημαντική μείωση φόρτου και χρόνου I/O.

- Μείωση των logical reads από 2601 σε 508 (περίπου 80%).
- Μείωση του χρόνου εκτέλεσης κατά περίπου 57%.
- Το execution plan βελτιώθηκε, καθώς ο SQL Server χρησιμοποίησε πιο στοχευμένες αναζητήσεις.

Ζήτημα Δεύτερο

Queries:

```sql
-- query 1
SELECT title FROM movies WHERE pyear BETWEEN 1990 AND 2000;

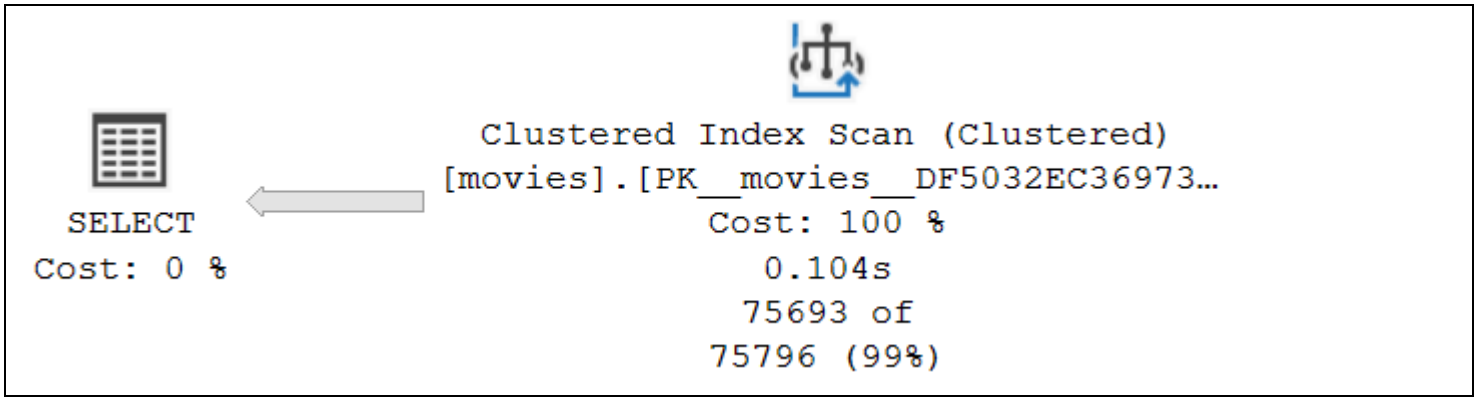
-- query 2
SELECT pyear, title FROM movies WHERE pyear BETWEEN 1990 AND 2000;

-- query 3
SELECT title, pyear FROM movies
WHERE pyear BETWEEN 1990 AND 2000
ORDER BY pyear, title;
```

Αρχικά εκτελέσαμε τα παραπάνω επερωτήματα χωρίς ευρετήριο και πήραμε τα παρακάτω δεδομένα:

Για το query 1:

Αρχικό Πλάνο Εκτέλεσης:



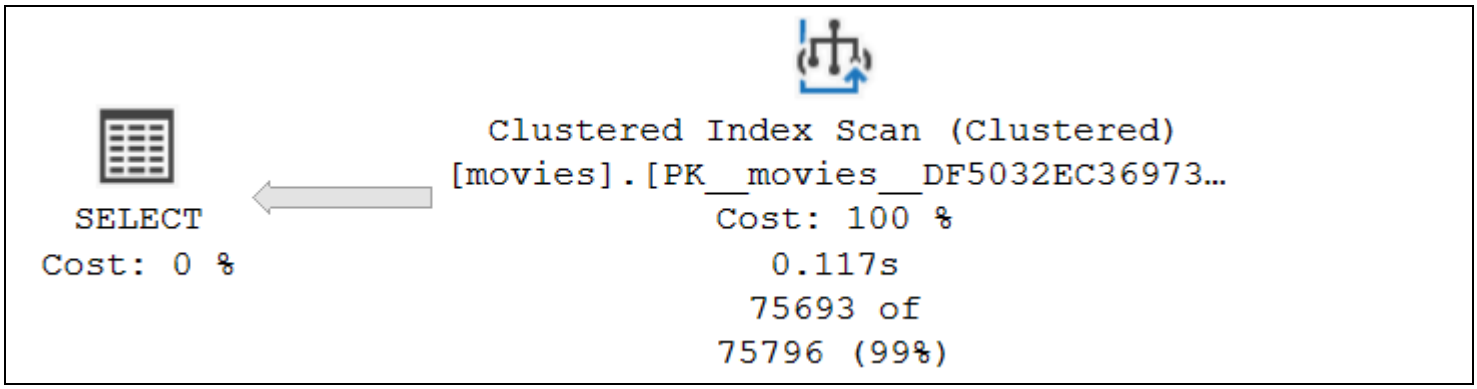
Αρχικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies	1	1918	2	1917

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	125	1037

Για το query 2:

Αρχικό Πλάνο Εκτέλεσης:



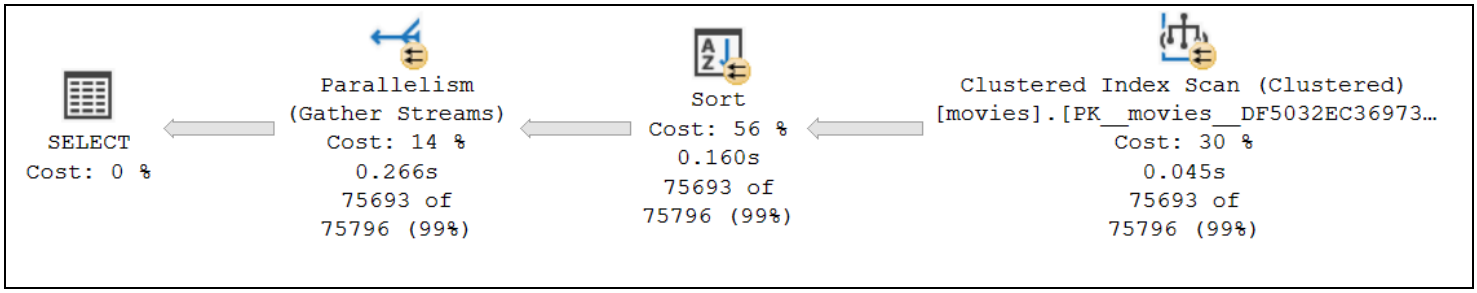
Αρχικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies	1	1918	2	1917

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	203	894

Για το query 3:

Αρχικό Πλάνο Εκτέλεσης:



Αρχικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies	5	2012	2	1917

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	827	1124

Από όλα τα παραπάνω μπορούμε να έχουμε τα επόμενα συγκεντρωτικά αποτελέσματα:

Query	Logical Reads	CPU Time (ms)	Elapsed Time (ms)	Access Method
Query 1	1918	125	1037	Table Scan
Query 2	1918	203	894	Table Scan
Query 3	2012	827	1124	Table Scan

Όλα τα επερωτήματα κάνουν πλήρη σάρωση (table scan), που είναι αναμενόμενο, αφού δεν υπάρχει κανένα ευρετήριο στο pyear ή title. Ο πίνακας movies έχει primary key το mid οπότε και clustered index σε αυτό το γνώρισμα που δεν μας βοηθάει στα επερωτήματα που θέλουμε να εκτελέσουμε.

Ευρετήριο:

Σε όλα τα επερωτήματα η αναζήτηση γίνεται στο όρισμα pyear και επιλογή στο title ή και pyear οπότε θα πρέπει να δημιουργήσουμε ένα ευρετήριο στα ορίσματα pyear, title (pyear πρώτο γιατί χρησιμοποιείται για την αναζήτηση)

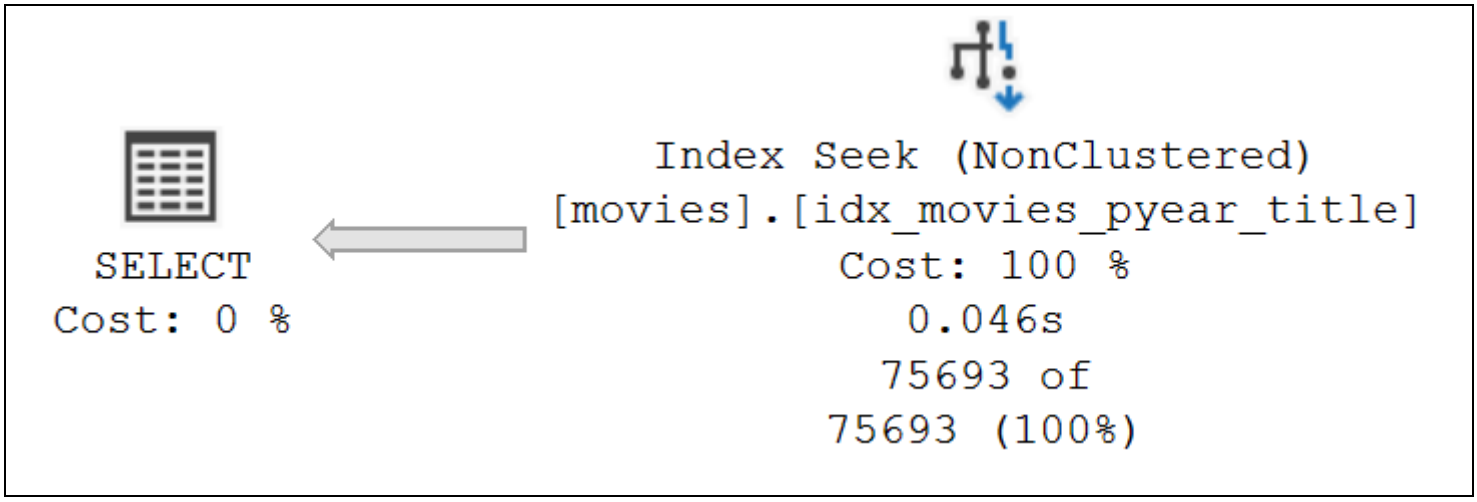
Η εντολή δημιουργίας του ευρετηρίου είναι η ακόλουθη:

```
CREATE INDEX idx_movies_pyear_title
ON movies(pyear, title);
```

Στην συνέχεια μετά την δημιουργία του παραπάνω ευρετηρίου ξαναεκτελούμε τα επερωτήματα και παίρνουμε:

Για το query 1:

Τελικό Πλάνο Εκτέλεσης:



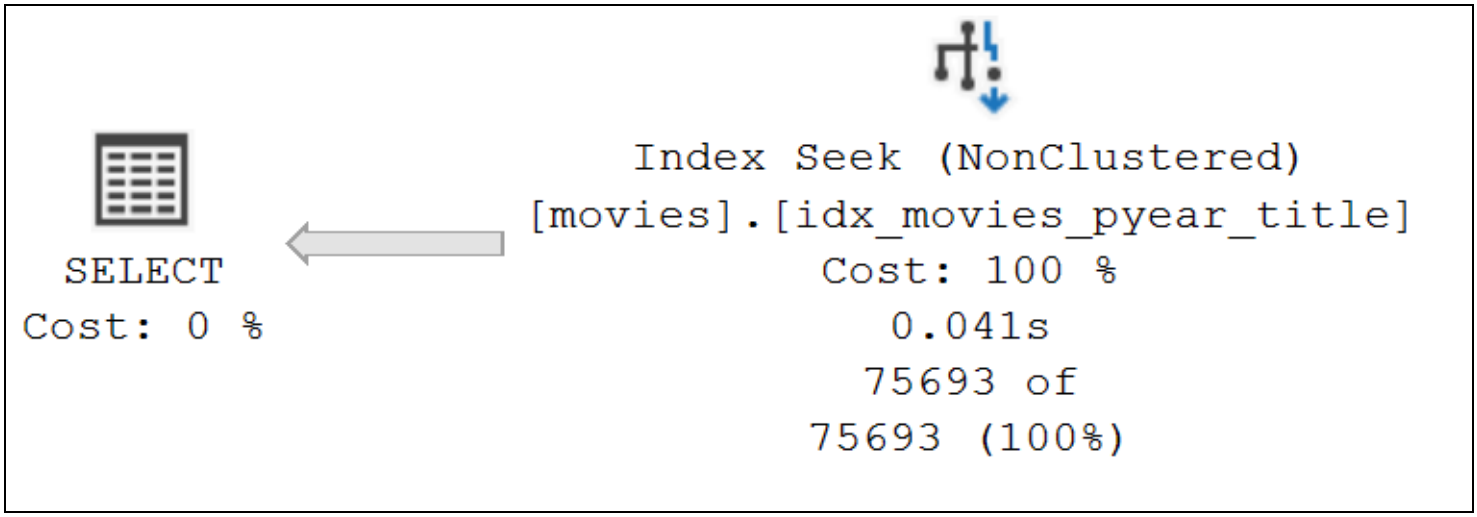
Τελικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies	1	351	2	361

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	63	794

Για το query 2:

Τελικό Πλάνο Εκτέλεσης:



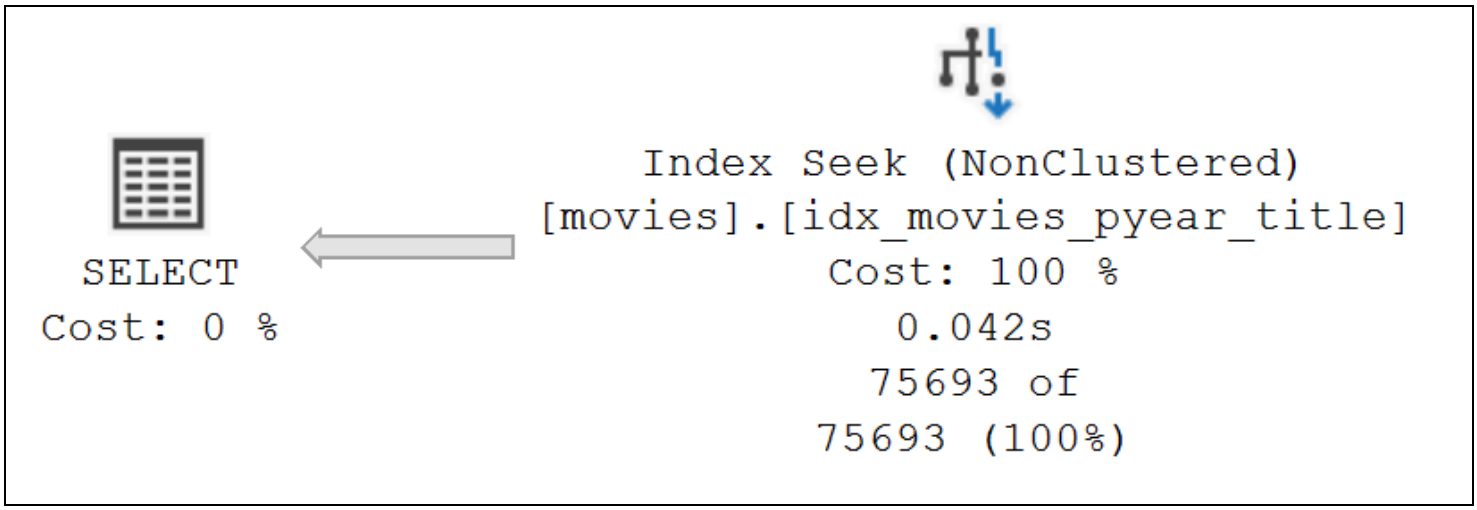
Τελικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies	1	351	2	361

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	32	586

Για το query 3:

Τελικό Πλάνο Εκτέλεσης:



Τελικά Στατιστικά:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies	1	351	2	361

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	47	569

Από όλα τα παραπάνω μπορούμε να έχουμε τα επόμενα τελικά συγκεντρωτικά αποτελέσματα:

Query	Logical Reads	CPU Time (ms)	Elapsed Time (ms)	Access Method
Query 1	351	63	794	Index Seek
Query 2	351	32	586	Index Seek
Query 3	351	47	569	Index Seek

Συμπεράσματα:

Συγκρίνοντας τα τελικά με τα αρχικά αποτελέσματα, βλέπουμε ότι τα logical reads μειώθηκαν από ~1900 σε 351 για όλα τα επερωτήματα, κάτι που δείχνει ότι το ευρετήριο αποφεύγει τη σάρωση του πίνακα και επιτελεί Index Seek στο ευρετήριο που φτιάξαμε και περιορίζει σημαντικά το I/O κόστος.

Η δημιουργία του ευρετηρίου idx_movies_pyear_title πέτυχε τον στόχο της επιτάχυνσης και των τριών ερωτημάτων.

Το pyear ως πρώτο πεδίο επιτρέπει αποδοτική αναζήτηση, ενώ η σειρά (pyear, title) επιτρέπει συσχέτιση με την ταξινόμηση στο **τρίτο ερώτημα**, ελαχιστοποιώντας την ανάγκη για επιπλέον sort operations.

Ζήτημα Τρίτο

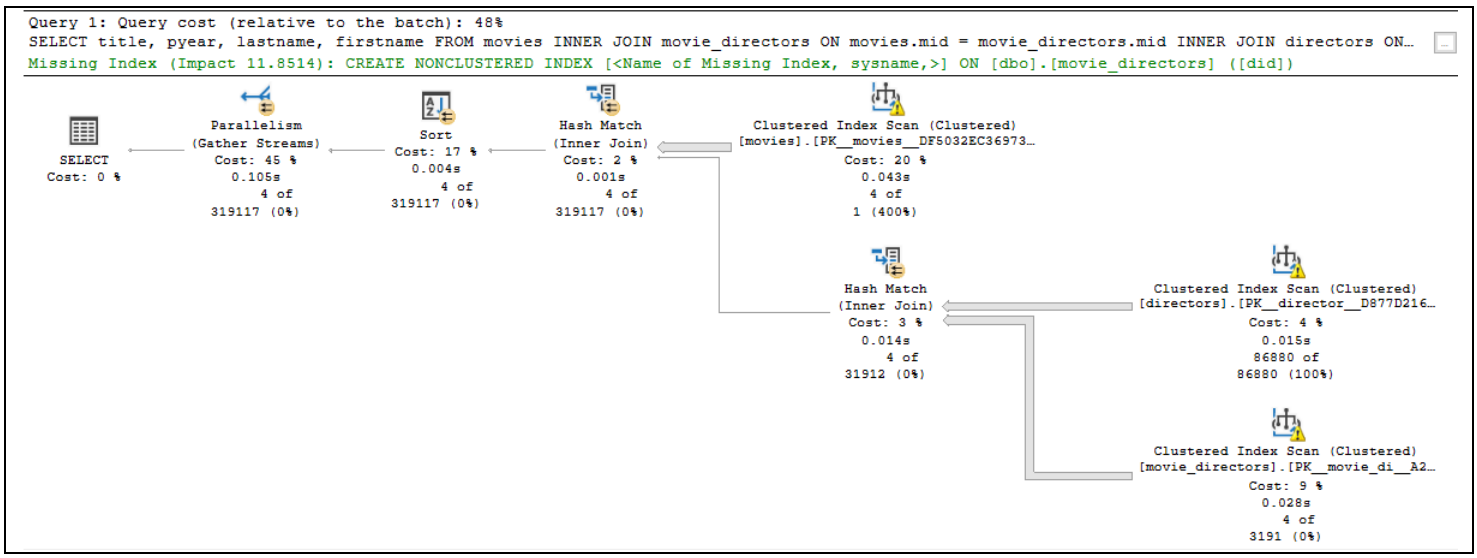
Queries:

```sql
-- Query 1: με βάση τον τίτλο της ταινίας
SELECT title, pyear, lastname, firstname
FROM movies
INNER JOIN movie_directors ON movies.mid = movie_directors.mid
INNER JOIN directors ON movie_directors.did = directors.did
WHERE title = 'Dreamer'
ORDER BY pyear;

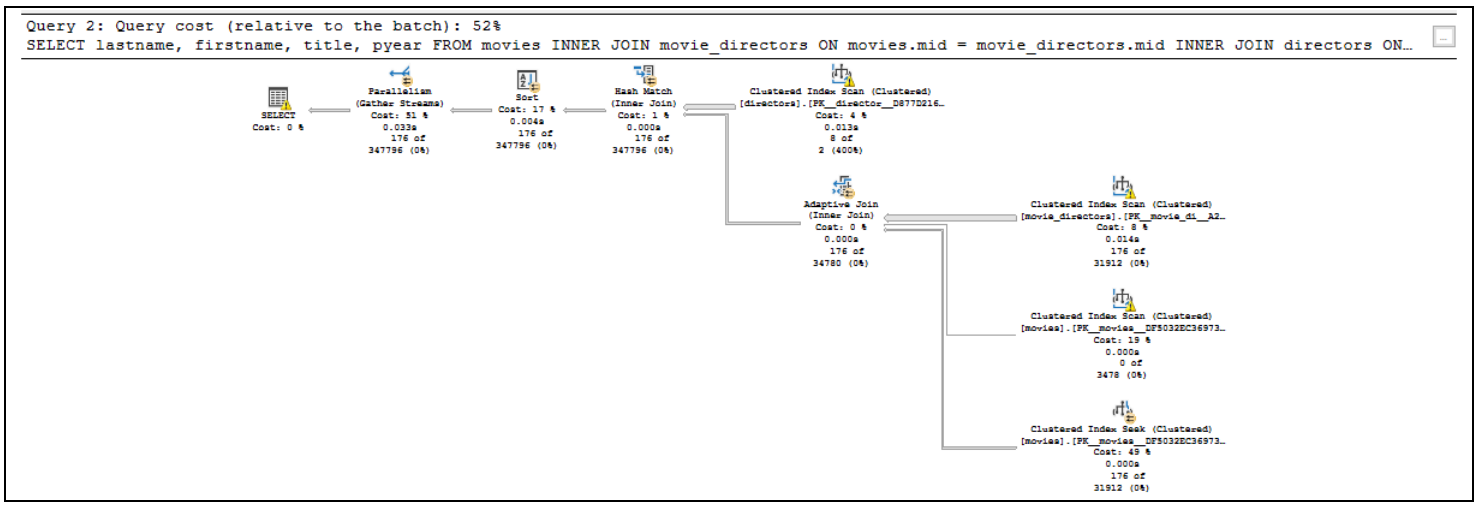
-- Query 2: με βάση το επώνυμο του σκηνοθέτη
SELECT lastname, firstname, title, pyear
FROM movies
INNER JOIN movie_directors ON movies.mid = movie_directors.mid
INNER JOIN directors ON movie_directors.did = directors.did
WHERE lastname = 'Nolan'
ORDER BY lastname, firstname;
```

Αρχικά εκτελέσαμε τα παραπάνω επερωτήματα χωρίς ευρετήριο και πήραμε τα παρακάτω δεδομένα:

Αρχικά Πλάνα Εκτέλεσης:
Για το query 1:



Για το query 2:



Αρχικά Στατιστικά:

Για το query 1:

Table	Scan count	logical reads	physical reads	read-ahead reads
movie_directors	5	708	2	678
directors	5	1048	1	356
movies	5	2012	2	1917

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	298	166

Για το query 2:

Table	Scan count	logical reads	physical reads	read-ahead reads
movie_directors	5	708	0	0
directors	5	1048	0	0
movies	0	528	0	0

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	76	287

Ευρετήρια:

Για την επιτάχυνση της εκτέλεσης των παραπάνω επερωτημάτων δημιουργήθηκαν τα εξής ευρετήρια:

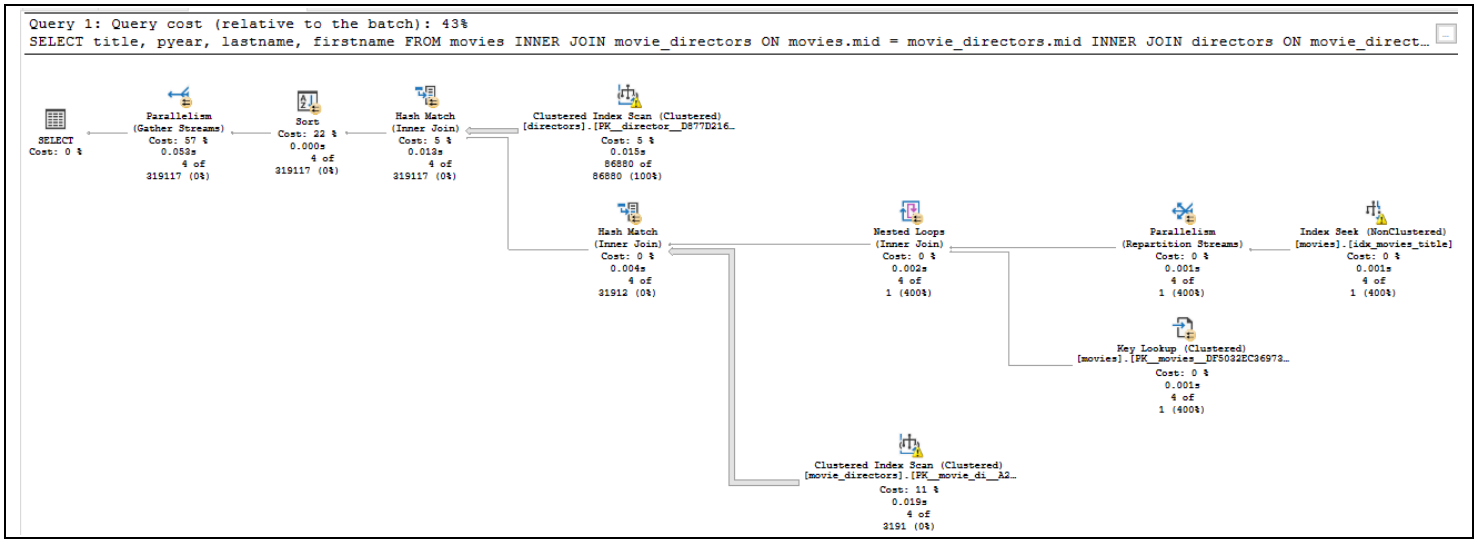
```
CREATE INDEX idx_movies_title
ON movies(title)

CREATE INDEX idx_directors_lastname
ON directors(lastname)
```

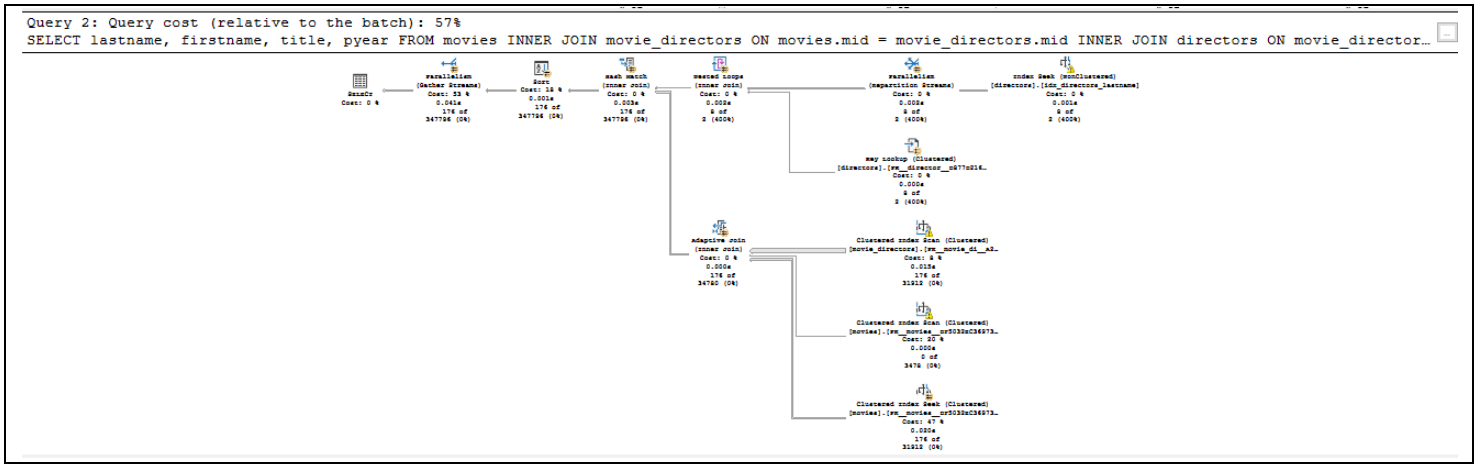
Η επιλογή βασίστηκε στα φίλτρα αναζήτησης (WHERE), με στόχο τη μετάβαση από table scan σε index seek και την ελαχιστοποίηση του I/O. Στην συνέχεια μετά την δημιουργία των παραπάνω ευρετηρίων ξαναεκτελούμε τα επερωτήματα και παίρνουμε:

Τελικά Πλάνα Εκτέλεσης:

Για το query 1:



Για το query 2:



Τελικά Στατιστικά:

Για το query 1:

Table	Scan count	logical reads	physical reads	read-ahead reads
movie_directors	5	708	2	678
directors	5	1048	1	356
movies	5	18	2	8

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	187	93

Για το query 2:

Table	Scan count	logical reads	physical reads	read-ahead reads
movie_directors	5	708	0	0
directors	5	20	1	8
movies	0	528	115	0

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	95	276

Συμπεράσματα:

Η χρήση ευρετηρίων μείωσε σημαντικά τα I/O κόστη στους βασικούς πίνακες:

Στο query 1: Τα logical reads στο movies μειώθηκαν από 2012 σε 18.

Στο query 2: Στο directors μειώθηκαν από 1048 σε 20.

Η βελτίωση είναι αποτέλεσμα της χρήσης Index Seek, όπως φαίνεται και από τα πλάνα εκτέλεσης. Ο χρόνος εκτέλεσης μειώθηκε κατά 40-50% στο Query 1, ενώ στο Query 2 υπήρξε βελτίωση σε I/O αλλά μικρότερη διαφορά στον συνολικό χρόνο — ενδεχομένως λόγω πλήθους επιστρεφόμενων εγγραφών.

Ζήτημα Τέταρτο

Α.

Queries:

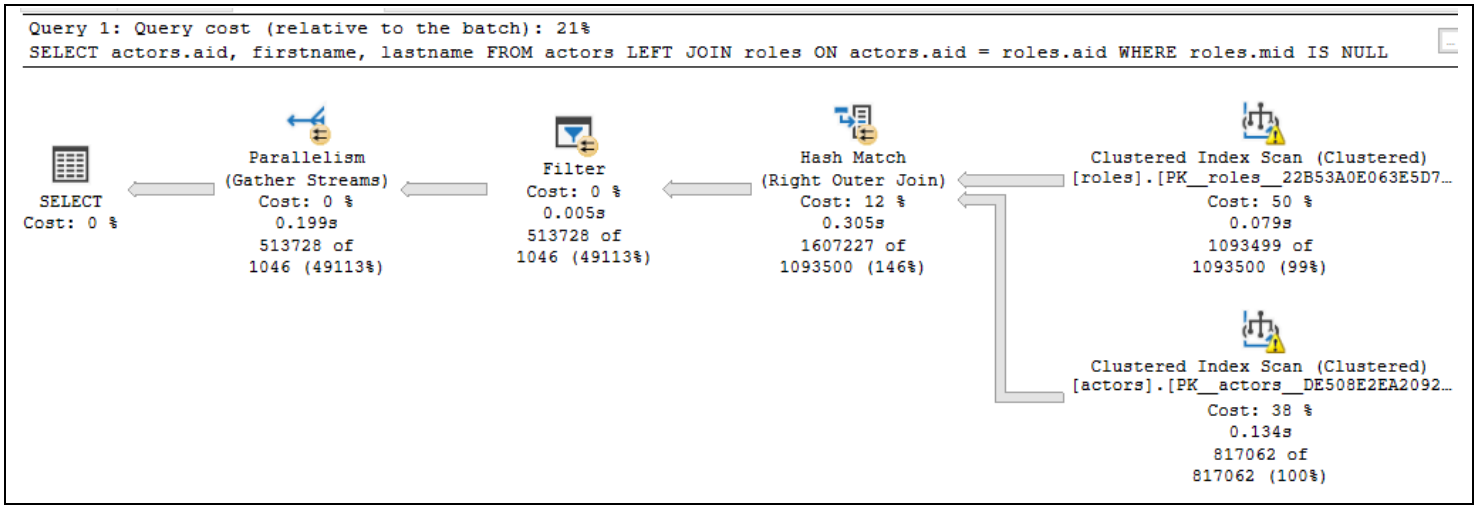
```sql
-- Query 1: ηθοποιοί χωρίς καμία ταινία
SELECT actors.aid, firstname, lastname
FROM actors
LEFT JOIN roles ON actors.aid = roles.aid
WHERE roles.mid IS NULL;

-- Query 2: ρόλοι συγκεκριμένου ηθοποιού
SELECT actors.aid, lastname, firstname, a_role, title
FROM actors
INNER JOIN roles ON actors.aid = roles.aid
INNER JOIN movies ON roles.mid = movies.mid
WHERE lastname = 'Paez' and firstname = 'Alex'
ORDER BY lastname, firstname, actors.aid;
```

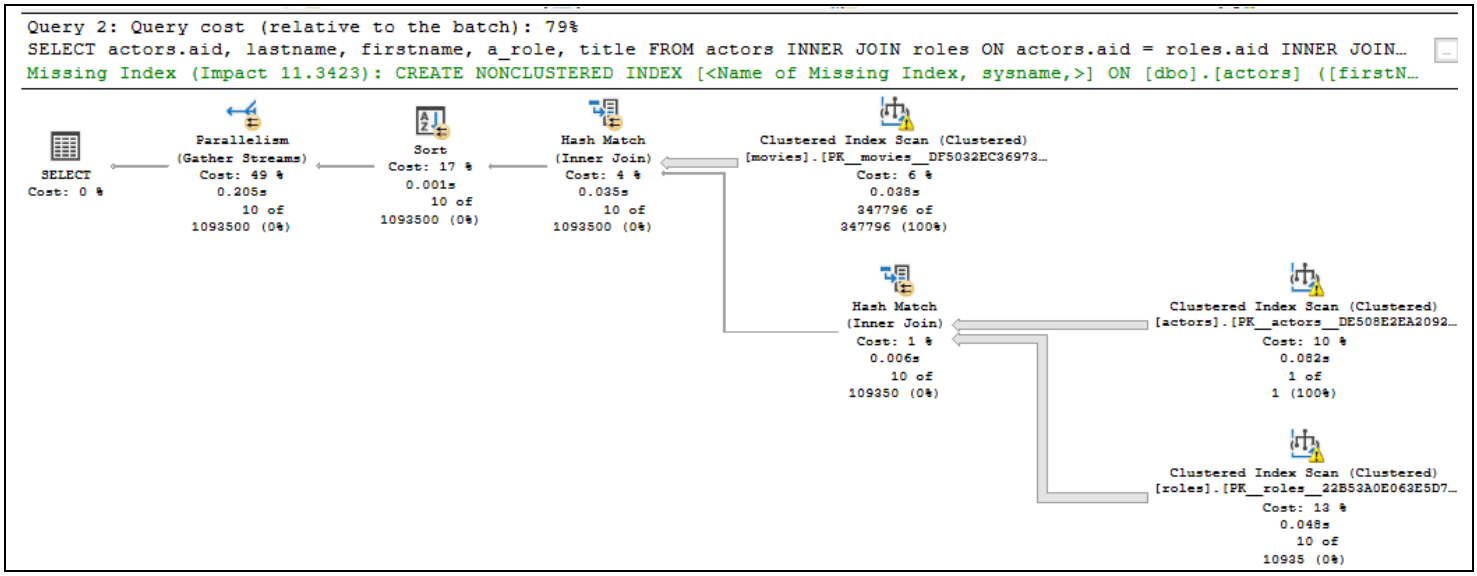
Αρχικά εκτελέσαμε τα παραπάνω επερωτήματα χωρίς ευρετήριο και πήραμε τα παρακάτω δεδομένα:

Αρχικά Πλάνα Εκτέλεσης:

Για το query 1:



Για το query 2:



Αρχικά Στατιστικά:

Για το query 1:

Table	Scan count	logical reads	physical reads	read-ahead reads
actors	5	3446	2	3362
roles	5	4489	1	4420

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	1875	4718

Για το query 2:

Table	Scan count	logical reads	physical reads	read-ahead reads
actors	5	3530	0	0
roles	5	4489	0	0
movies	5	2012	2	1917

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	812	399

Ευρετήρια:

Για την επιτάχυνση της εκτέλεσης των παραπάνω επερωτημάτων δημιουργήθηκαν τα εξής ευρετήρια:

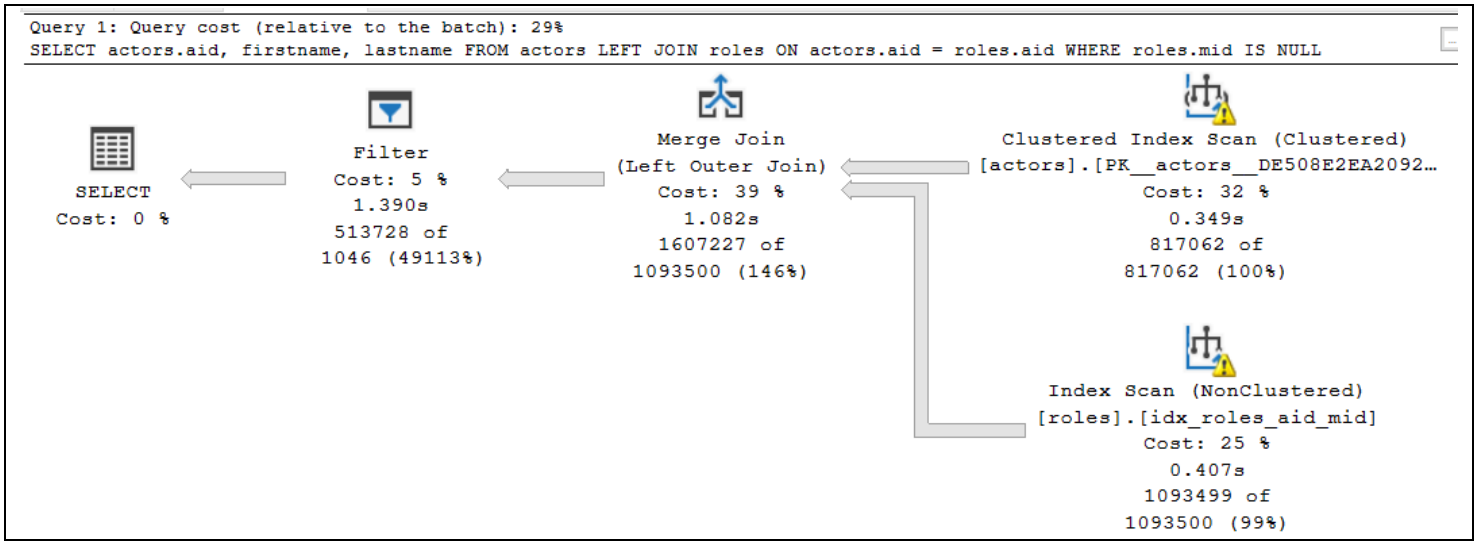
```
-- Για το Query 1: επιτάχυνση του LEFT JOIN
CREATE INDEX idx_roles_aid
ON roles(aid);

-- Για το Query 2: επιτάχυνση φίλτρου και ταξινόμησης
CREATE INDEX idx_actors_name
ON actors(lastname, firstname);

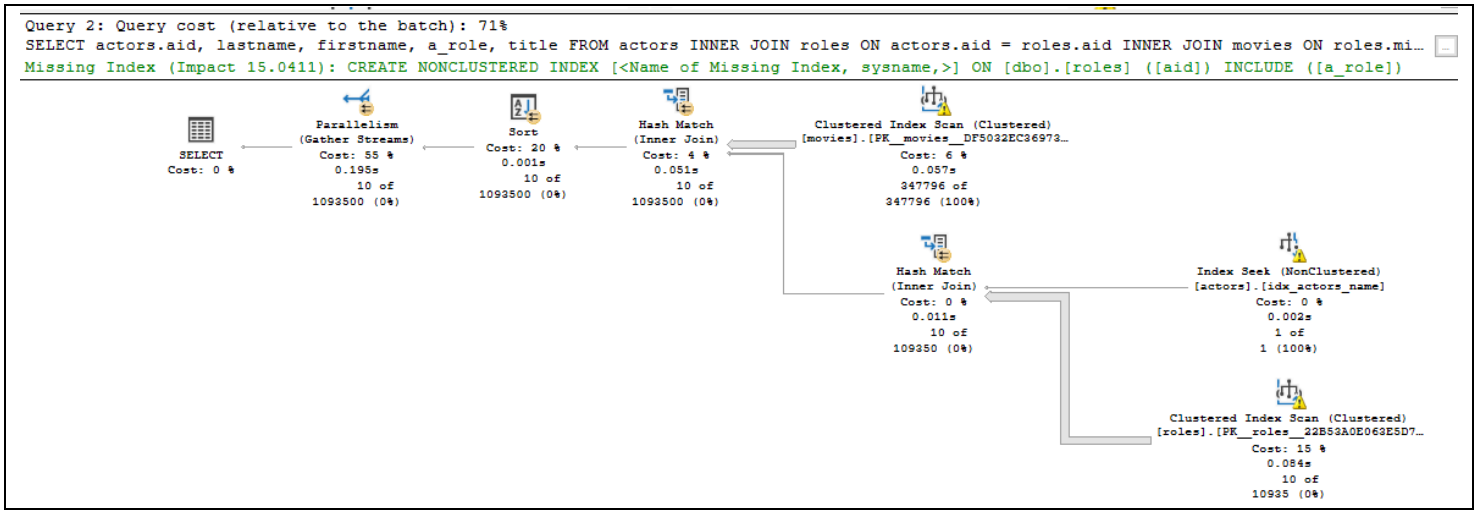
-- Για το Query 2: επιτάχυνση των JOINs με aid και mid
CREATE INDEX idx_roles_aid_mid
ON roles(aid, mid);
```

Στην συνέχεια μετά την δημιουργία των παραπάνω ευρετηρίων ξαναεκτελούμε τα επερωτήματα και παίρνουμε:

Τελικά Πλάνα Εκτέλεσης:
Για το query 1:



Για το query 2:



Τελικά Στατιστικά:

Για το query 1:

Table	Scan count	logical reads	physical reads	read-ahead reads
actors	1	3361	2	3362
roles	1	1900	3	1910

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	2093	5808

Για το query 2:

Table	Scan count	logical reads	physical reads	read-ahead reads
actors	4	6	2	8
roles	5	4489	1	4420
movies	5	2012	2	1917

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	610	528

Συμπεράσματα:

Σύγκριση Απόδοσης

Query 1 (LEFT JOIN)

- roles: από 4489 → 1900 logical reads
- actors: μικρή διαφορά (3446 → 3361)
- CPU time αυξήθηκε: 1875 → 2093 ms
- Το query παραμένει βαρύ λόγω πολύ μεγάλου αποτελέσματος (~513k εγγραφές).

Query 2 (αναζήτηση συγκεκριμένου ηθοποιού)

- actors: από 3530 → 6
- roles, movies: σταθερά
- CPU time βελτιώθηκε: 812 → 610 ms

Το idx_actors_name επέτρεψε την αποδοτική χρήση Index Seek στο φίλτρο lastname='Paez' AND firstname='Alex'. Αντιθέτως, το idx_roles_aid μείωσε τα I/O στο Query 1, αλλά δεν οδήγησε σε συνολική βελτίωση χρόνου λόγω του μη επιλεκτικού χαρακτήρα του LEFT JOIN (δεν περιορίζει αρκετά τα αποτελέσματα ειδικά στην περίπτωσή μας επιστρέφει 513729 εγγραφές από όλες που είναι 817062, άρα το query επιστρέφει πάνω από 62% των εγγραφών του πίνακα actors.

Τα ευρετήρια που επιλέχθηκαν κρίνονται κατάλληλα με βάση τις απαιτήσεις των queries, και επιτυγχάνουν σημαντική βελτίωση όπου αυτό είναι εφικτό.

Ζήτημα Τέταρτο

Β. Ένας άλλος τρόπος για να γράψουμε το πρώτο επερώτημα ώστε να εκτελείται με πιο αποδοτικό τρόπο είναι ο εξής:

```sql
-- Βελτιστοποιημένη μορφή με NOT EXISTS
SELECT a.aid, a.firstname, a.lastname
FROM actors a
WHERE NOT EXISTS (
    SELECT *
    FROM roles r
    WHERE r.aid = a.aid

);
```

Έτσι έχουμε για το αρχικό επερώτημα:

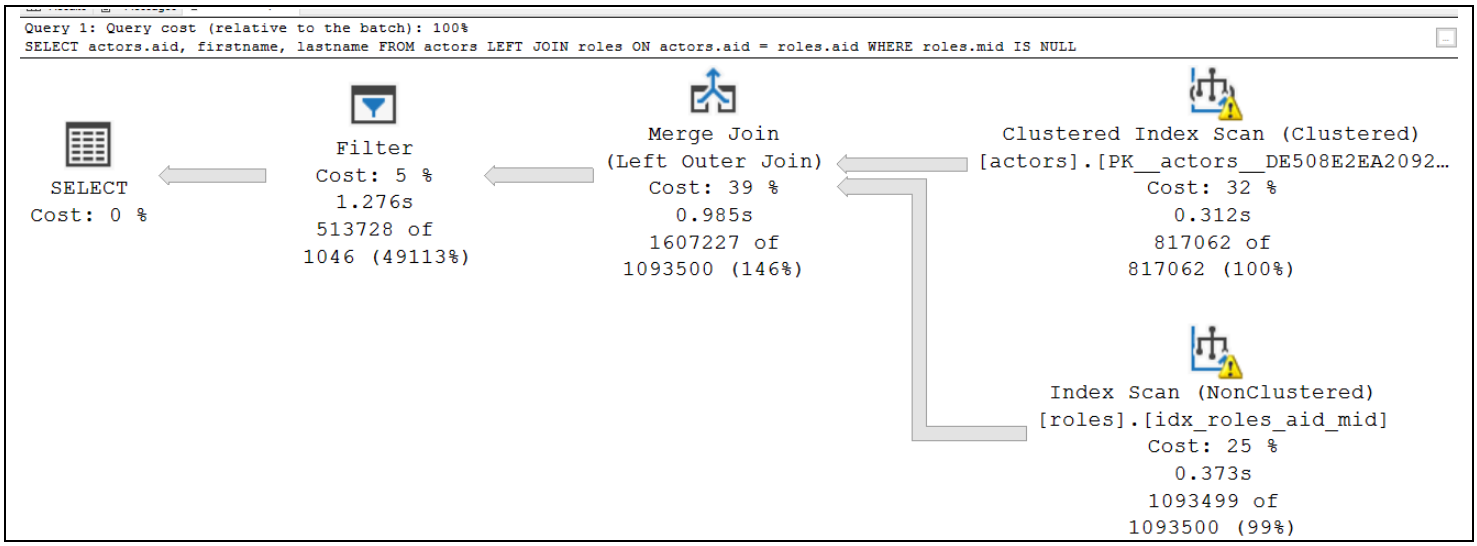


Table	Scan count	logical reads	physical reads	read-ahead reads
actors	1	3361	2	3362
roles	1	1900	3	1910

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	1937	5600

Και για το βελτιωμένο:

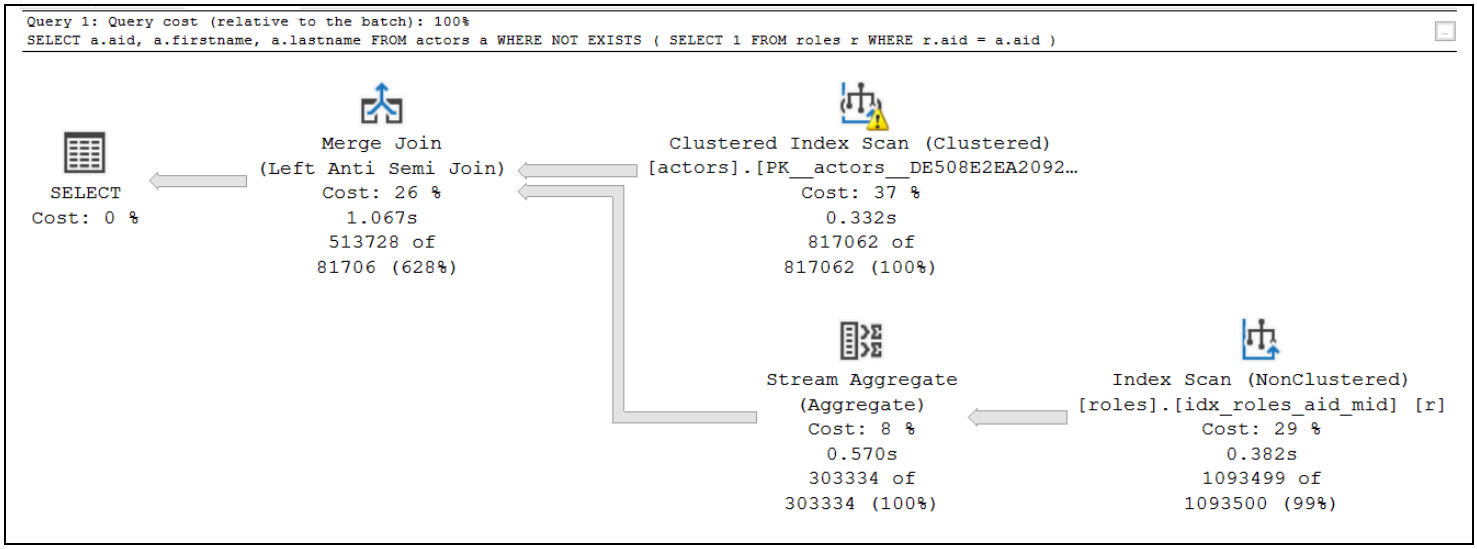


Table	Scan count	logical reads	physical reads	read-ahead reads
actors	1	3361	2	3362
roles	1	1900	3	1910

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	1578	5607

Σύγκριση Απόδοσης επερωτημάτων:

Σύνταξη	CPU Time	Elapsed Time	Logical Reads (actors / roles)
LEFT JOIN + IS NULL	1937 ms	5600 ms	3361 / 1900
NOT EXISTS	1578 ms	5607 ms	3361 / 1900

Συμπεράσματα:

Παρόλο που τα δύο επερωτήματα επιστρέφουν το ίδιο αποτέλεσμα και έχουν τον ίδιο αριθμό αναγνώσεων, η εναλλακτική σύνταξη με NOT EXISTS επιτυγχάνει μειωμένο χρόνο CPU, υποδεικνύοντας πιο αποδοτικό πλάνο εκτέλεσης.

Ζήτημα Πέμπτο

Queries:

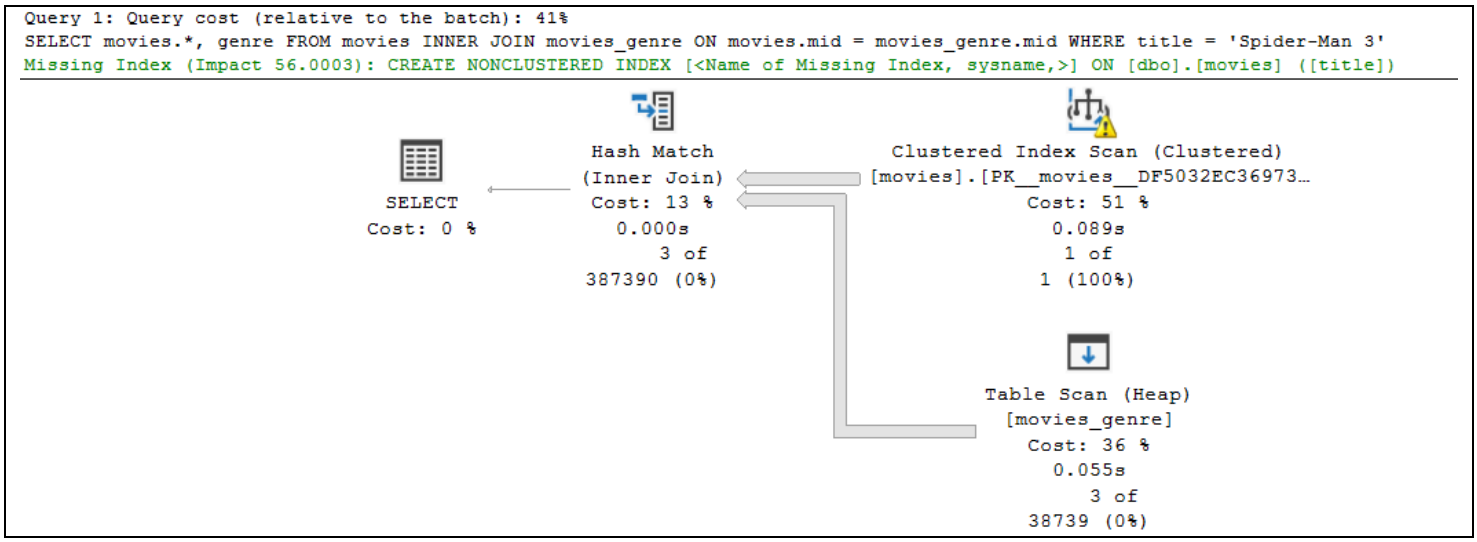
```sql
-- Query 1
SELECT AVG(rating) AS average_rating
FROM movies_genre
INNER JOIN user_movies ON movies_genre.mid = user_movies.mid
WHERE genre = 'Drama';

-- Query 2
SELECT movies.*, genre
FROM movies
INNER JOIN movies_genre ON movies.mid = movies_genre.mid
WHERE title = 'Spider-Man 3';
```

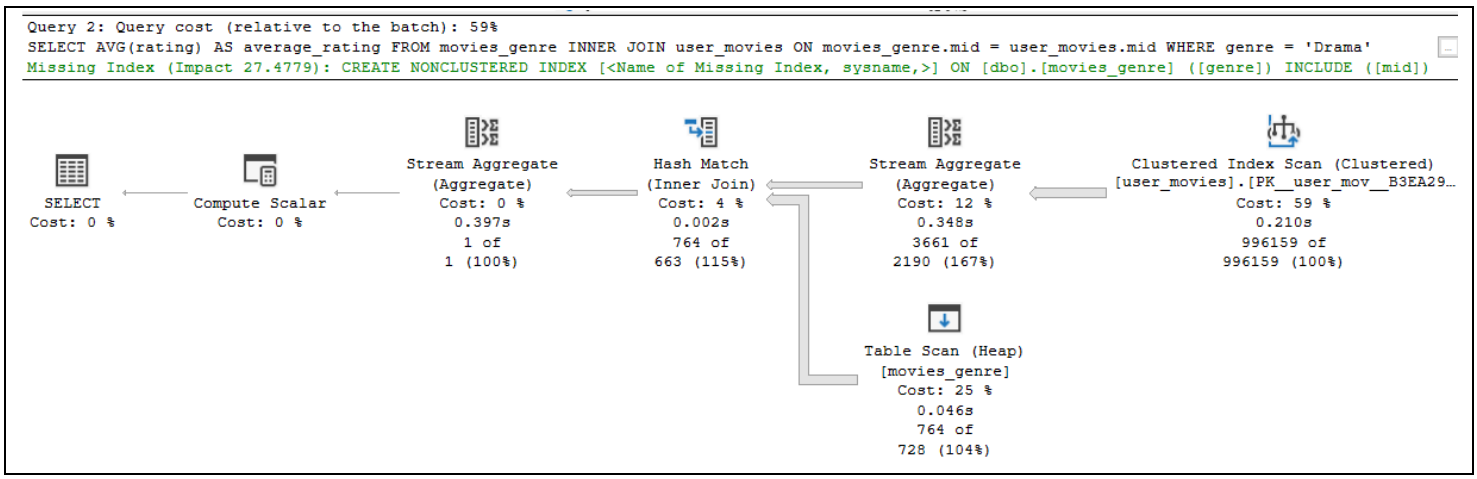
Αφού η συχνότητα εκτέλεσης του πρώτου επερωτήματος είναι 20% και του δεύτερου 80%, το πιο πιθανό είναι όταν έχει η ώρα να εκτελεστεί το πρώτο επερώτημα, θα έχει ήδη εκτελεστεί το δεύτερο επερώτημα. Αρχικά λοιπόν εκτελέσαμε τα παραπάνω επερωτήματα σε δέσμη με πρώτο το δεύτερο και δεύτερο το πρώτο επερώτημα, χωρίς ευρετήριο και πήραμε τα παρακάτω δεδομένα:

Αρχικά Πλάνα Εκτέλεσης:

Για το query 2:



Για το query 1:



Αρχικά Στατιστικά:

Για το query 2:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies_genre	1	1123	0	1123
movies	1	1918	2	1917

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	141	179

Για το query 1:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies_genre	1	1123	0	0
user_movies	1	2601	2	2603

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	375	559

Παρατηρούμε ότι οι πίνακες διαβάζονται με Table Scan, άρα και τα δύο queries σαρώνουν ολόκληρους τους πίνακες και δεν επωφελούνται από καμία βελτιστοποίηση. Το Query 2 εκτελείται πιο γρήγορα γιατί είναι πιο επιλεκτικό, αλλά τρέχει 4 φορές συχνότερα, άρα έχει μεγαλύτερη επιρροή στον συνολικό φόρτο.

Πρωτεύον κλειδί και Ευρετήρια:

Για την επιτάχυνση της εκτέλεσης των παραπάνω επερωτημάτων αρχικά θα πρέπει να δημιουργήσουμε ένα πρωτεύον κλειδί στον πίνακα movies_genre. Ο πίνακας movies_genre(mid, genre) δεν έχει PK, έτσι η φυσική επιλογή για PK είναι το ζεύγος (mid, genre), γιατί κάθε ταινία έχει μοναδικά genres:

```
ALTER TABLE movies_genre
ADD CONSTRAINT pk_movies_genre PRIMARY KEY (mid, genre);
```

Για το query 1 (WHERE genre = 'Drama'), θέλουμε να εντοπίζουμε όλα τα mid με genre = 'Drama' και να τα συσχετίζουμε με user_movies (JOIN on mid). Δεν χρειαζόμαστε άλλα πεδία από τον πίνακα movies_genre, άρα:

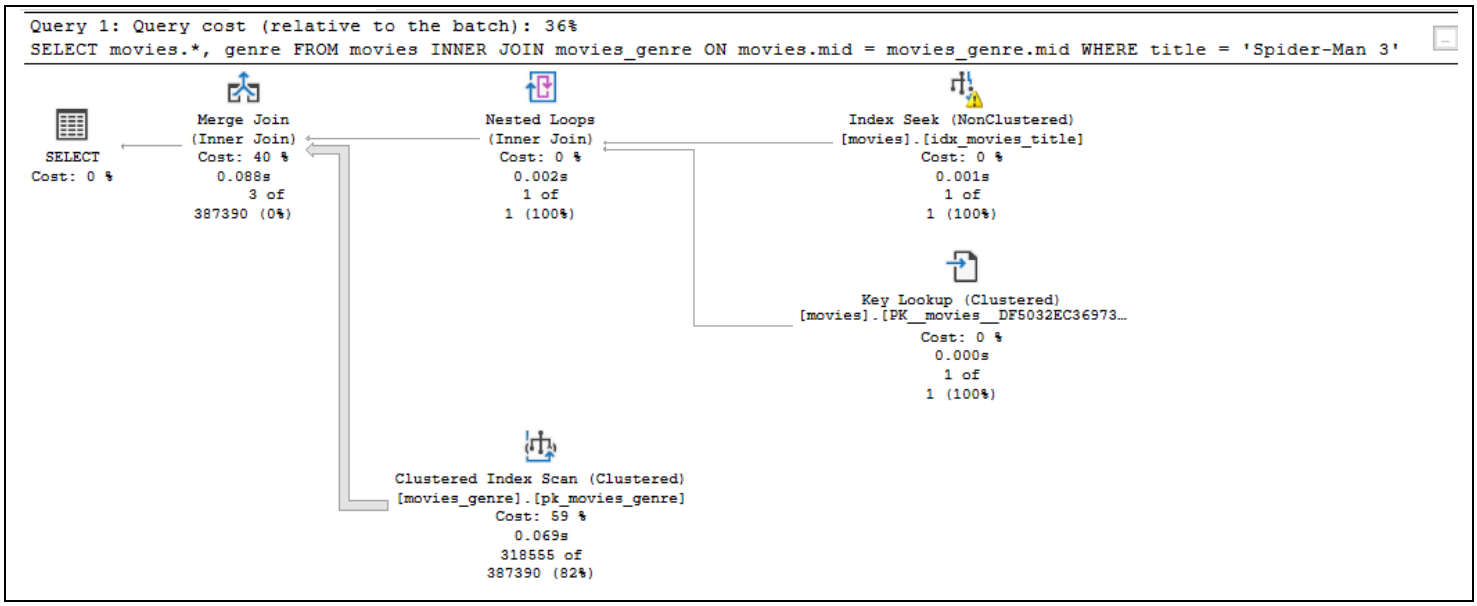
```
CREATE INDEX idx_movies_genre_genre
ON movies_genre(genre);
```

Για το query 2 (WHERE title = 'Spider-Man 3'), θέλουμε να εντοπίζουμε movies από title, JOIN με movies_genre → βάση του mid και να επιστρέφονται όλα τα πεδία του movies + genre, άρα:

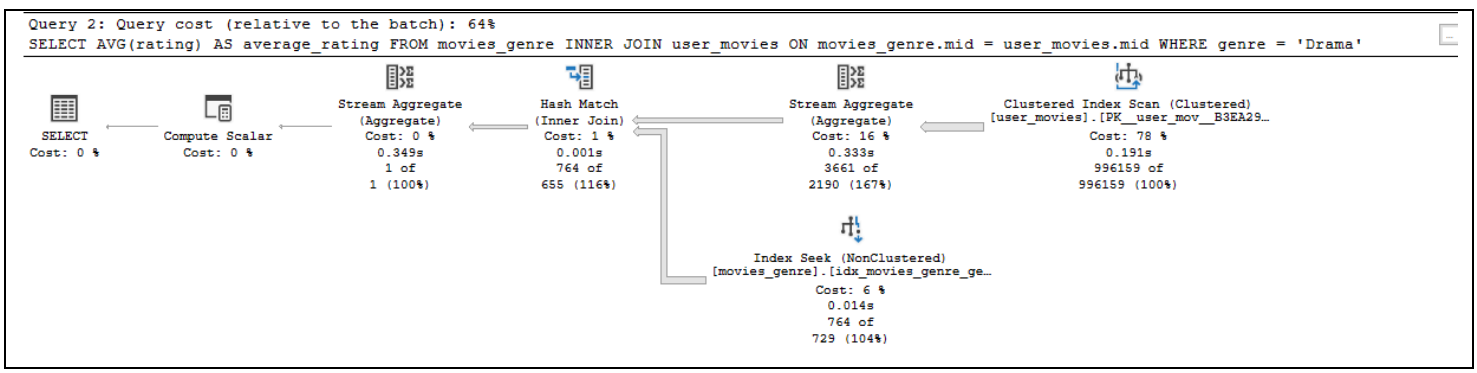
```
CREATE INDEX idx_movies_title
ON movies(title);
```

Στην συνέχεια μετά την δημιουργία πρωτεύοντος κλειδιού και των παραπάνω ευρετηρίων ξαναεκτελούμε τα επερωτήματα (ως δέσμη με την ίδια σειρά) και παίρνουμε:

Τελικά Πλάνα Εκτέλεσης:
Για το query 2:



Για το query 1:



Τελικά Στατιστικά:

Για το query 2:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies_genre	1	931	3	1148
movies	1	6	5	0

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	94	116

Για το query 1:

Table	Scan count	logical reads	physical reads	read-ahead reads
movies_genre	1	177	4	179
user_movies	1	2601	2	2603

Action	CPU time (ms)	elapsed time (ms)
SQL Server Execution Times:	328	522

Συμπεράσματα:

Η χρήση πρωτεύοντος κλειδιού και στοχευμένων ευρετηρίων οδήγησε σε σημαντική μείωση του κόστους εκτέλεσης:

- Ο πίνακας movies στο Query 2 (που εκτελείται πιο συχνά) αξιοποίησε Index Seek, μειώνοντας τα reads από 1918 σε 6.
- Ο πίνακας movies_genre βελτιώθηκε αισθητά και στα δύο queries (1123 → 931 στο Query 2, και 1123 → 177 στο Query 1).
- Το Query 1 παρέμεινε βαρύ λόγω του πίνακα user_movies, που δεν ευρετηριάστηκε.

Η συνολική απόδοση του συστήματος ωφελείται σημαντικά από τις συγκεκριμένες παρεμβάσεις, ενώ δίνεται προτεραιότητα στο πιο συχνά εκτελούμενο ερώτημα όπως απαιτεί η εκφώνηση.